

ARTIGO DE PESQUISA/RESEARCH PAPER

Plataforma Web com Inteligência Artificial para Identificação de Doenças em Plantas

Web platform with Artificial Intelligence for identifying plant diseases

Maria Quaresma [Centro Federal de Educação Tecnológica Celso Sucknow da Fonseca | maria.rocha@aluno.cefet-rj.br]
Neemias Cruz [Centro Federal de Educação Tecnológica Celso Sucknow da Fonseca | neemiascruz@hotmail.com]
Diego Castro [Centro Federal de Educação Tecnológica Celso Sucknow da Fonseca | diego.castro@cefet-rj.br]

[Centro Federal de Educação Tecnológica Celso Sucknow da Fonseca, R. Miguel Ângelo, 96 - Maria da Graça, Rio de Janeiro - RJ, 20785-220, Brasil.]

Resumo. Este trabalho apresenta o desenvolvimento de uma plataforma web baseada em Inteligência Artificial para a identificação de doenças em plantas por meio da análise de imagens de folhas. A metodologia utilizada é uma abordagem expositiva, a solução foi implementada em arquitetura cliente-servidor, utilizando FastAPI, JavaScript, PostgreSQL e Docker, e integra dois modelos de aprendizado profundo baseados na EfficientNetB0, responsáveis pela identificação da espécie vegetal e da doença associada. Os modelos foram treinados com imagens da base PlantVillage, utilizando uma abordagem de divisão hold-out entre conjuntos de treinamento e validação, visando aumentar a consistência dos diagnósticos. Além disso, a plataforma oferece autenticação de usuários, armazenamento do histórico de análises e geração automática de informações textuais complementares. Os resultados demonstraram a viabilidade da abordagem proposta, evidenciando seu potencial de aplicação no contexto da agricultura digital e do diagnóstico fitossanitário.

Abstract. This work presents the development of a web platform based on Artificial Intelligence for the identification of plant diseases through the analysis of leaf images. The methodology employs an expository approach, the solution was implemented in a client-server architecture, using FastAPI, JavaScript, PostgreSQL, and Docker, and integrates two deep learning models based on EfficientNetB0, responsible for identifying the plant species and the associated disease. The models were trained with images from the PlantVillage database, using a hold-out split approach between training and validation sets, increasing the consistency of diagnoses. In addition, the platform offers user authentication, storage of analysis history, and automatic generation of complementary textual information. The results demonstrated the viability of the proposed approach, highlighting its potential application in the context of digital agriculture and phytosanitary diagnosis.

Palavras-chave: Visão Computacional, Aprendizado Profundo, Agricultura Digital

Keywords: Computer Vision, Deep Learning, Digital Agriculture

1 Introdução

A agricultura desempenha papel fundamental na economia mundial e na segurança alimentar, sendo responsável pelo fornecimento de alimentos, matérias-primas e recursos essenciais para a sociedade. Entretanto, doenças que afetam culturas agrícolas representam uma das principais causas de perdas produtivas, comprometendo tanto a qualidade quanto a quantidade das colheitas. Nesse contexto, a identificação precoce de doenças em plantas torna-se um fator determinante para reduzir prejuízos econômicos e promover práticas agrícolas mais sustentáveis. Com o avanço das tecnologias digitais, soluções baseadas em Inteligência Artificial (IA) e visão computacional vêm sendo exploradas como alternativas para auxiliar agricultores e produtores no monitoramento das plantações [Mohanty *et al.*, 2016].

Nos últimos anos, o emprego de técnicas de Aprendizado Profundo (Deep Learning) tem proporcionado avanços significativos na classificação automática de doenças vegetais. Revisões recentes evidenciam que diferentes arquiteturas, como Redes Neurais Convolucionais (CNNs), Vision Transformers (ViTs) e modelos híbridos, apresentam elevados índices de desempenho em bases públicas de dados, contribuindo para a automação do diagnóstico fitossanitário e para o desenvolvimento da agricultura de precisão [Pacal *et al.*,

2024; Liu and Wang, 2021]. Nesse contexto, [Mohanty *et al.*, 2016] demonstraram que modelos baseados em redes neurais convolucionais podem alcançar taxas de acurácia superiores a 99% em conjuntos de dados controlados, evidenciando o potencial da Inteligência Artificial como ferramenta de apoio à identificação de doenças em plantas.

Entre as arquiteturas mais promissoras para tarefas de classificação de imagens destaca-se a EfficientNet, proposta por [Tan and Le, 2019]. Esse modelo introduziu uma estratégia de escalonamento composto capaz de balancear profundidade, largura e resolução da rede neural, proporcionando elevado desempenho com menor custo computacional quando comparado a arquiteturas tradicionais. Tais características tornam essa arquitetura particularmente adequada para aplicações voltadas à agricultura digital, nas quais são desejáveis rapidez, precisão e possibilidade de implantação em ambientes com recursos computacionais limitados.

Apesar dos avanços observados na literatura, diversos desafios ainda dificultam a aplicação prática dessas soluções em cenários reais. Muitos modelos são desenvolvidos e avaliados em ambientes controlados, utilizando imagens com iluminação adequada, fundo padronizado e elevada qualidade visual. Em condições reais, entretanto, fatores como variações ambientais, ruídos nas imagens, diferenças entre espécies vegetais e semelhanças entre determinadas doenças podem

comprometer a capacidade de generalização dos modelos. Revisões recentes apontam que a robustez e a confiabilidade dos sistemas inteligentes continuam sendo questões relevantes para a adoção dessas tecnologias em larga escala [Pacal *et al.*, 2024].

Além disso, grande parte dos trabalhos concentra-se na utilização de um único modelo de classificação, sem considerar mecanismos adicionais capazes de verificar a consistência entre a espécie vegetal identificada e a doença associada. Dessa forma, ainda existe uma lacuna relacionada à integração de múltiplos modelos especializados e à utilização de estratégias complementares de validação, capazes de reduzir ambiguidades e aumentar a confiabilidade dos diagnósticos produzidos [Shoaib *et al.*, 2023].

Diante desse cenário, o presente trabalho tem como objetivo desenvolver uma plataforma web inteligente para identificação automática de doenças em plantas por meio de técnicas de Inteligência Artificial e Visão Computacional. A solução proposta baseia-se em uma arquitetura cliente-servidor e combina dois modelos especializados, sendo um destinado à identificação da espécie vegetal e outro à classificação da doença. Adicionalmente, foram implementadas estratégias de validação dos resultados baseadas na análise de compatibilidade entre planta e doença e cálculo do nível de confiança das previsões, visando reduzir inconsistências e aumentar a confiabilidade dos resultados. A plataforma também contempla funcionalidades de autenticação de usuários, armazenamento do histórico de diagnósticos e geração automática de informações textuais complementares, constituindo uma solução integrada voltada ao apoio à tomada de decisão no contexto da agricultura digital [Quaresma, 2026].

2 Materiais e Métodos

Esta pesquisa caracteriza-se como aplicada, quantitativa e experimental, tendo como objetivo o desenvolvimento e a avaliação de uma plataforma web para identificação de doenças em plantas por meio da análise de imagens de folhas. A solução emprega técnicas de Visão Computacional e Aprendizado Profundo para apoiar o diagnóstico fitossanitário e contribuir para aplicações de agricultura digital e agricultura de precisão [Mohanty *et al.*, 2016; Pacal *et al.*, 2024].

No contexto deste trabalho, a Visão Computacional é aplicada ao processamento automático de imagens de folhas, permitindo que o sistema extraia informações visuais relevantes e identifique a espécie vegetal e a doença associada. De forma geral, um sistema de Visão Computacional é composto por quatro etapas principais: aquisição da imagem, pré-processamento, extração de características e interpretação dos resultados. Com a utilização de arquiteturas de Aprendizado Profundo, como a EfficientNetB0, a etapa de extração de características passa a ser realizada automaticamente pela rede neural, reduzindo a necessidade de engenharia manual de atributos [Szeliski, 2022].

A Tabela 1 relaciona esses quatro etapas com sua implementação na plataforma desenvolvida, indica como cada etapa do pipeline de Visão Computacional está presente na solução proposta.

Tabela 1. Relação entre as etapas da Visão Computacional e sua implementação no sistema proposto.

Etapas da Visão Computacional	Implementação no sistema proposto
Aquisição da imagem	Captura da fotografia da folha por meio do envio (<i>upload</i>) realizado pelo usuário na interface web.
Pré-processamento	Redimensionamento da imagem para 224×224 pixels e normalização utilizando a função <i>preprocess_input</i> da EfficientNetB0.
Extração de características	Extração automática de atributos visuais pela arquitetura EfficientNetB0, empregando Aprendizado Profundo, aprendizado por transferência (<i>Transfer Learning</i>) e ajuste fino (<i>Fine-Tuning</i>).
Interpretação e tomada de decisão	Classificação da espécie vegetal e da doença, validação da compatibilidade entre as previsões e cálculo do nível de confiança do diagnóstico.

2.1 Arquitetura e Implementação da Plataforma

A arquitetura do sistema foi concebida segundo o modelo cliente-servidor, sendo composta por uma interface web responsável pela interação com o usuário e por uma camada de serviços encarregada do processamento das requisições, persistência dos dados e integração com os modelos de Inteligência Artificial.

O sistema foi desenvolvido com a linguagem Python 3.11 na camada de backend, em razão de sua ampla adoção em aplicações de Ciência de Dados, Aprendizado de Máquina e Inteligência Artificial. Conforme destacado por [Raschka *et al.*, 2020], o ecossistema Python consolidou-se como uma das principais plataformas para o desenvolvimento de soluções voltadas à análise de dados e ao aprendizado de máquina, devido à disponibilidade de bibliotecas especializadas e à elevada produtividade proporcionada durante o desenvolvimento de software.

A organização interna da aplicação seguiu uma arquitetura em camadas, contemplando módulos responsáveis pelas rotas da API, autenticação dos usuários, serviços de negócio, persistência das informações e validação dos dados. Essa abordagem promove a separação de responsabilidades entre os componentes e favorece a modularidade, a reutilização de código e a manutenção do sistema. Segundo [Farias and Lazzari, 2023], arquiteturas baseadas em componentes bem definidos e baixo acoplamento contribuem para a construção de sistemas mais flexíveis e escaláveis.

A comunicação entre a interface web e a camada de serviços foi realizada por meio de uma API REST (Representational State Transfer), desenvolvida com o framework FastAPI e executada sobre o servidor ASGI Uvicorn. O FastAPI foi escolhido em virtude do suporte à programação assíncrona, da validação automática dos dados e do elevado desempenho no processamento das requisições. A adoção da arquitetura REST permite disponibilizar os recursos da aplicação por meio de endpoints acessados via HTTPS, favorecendo o baixo acoplamento entre os componentes e a interoperabilidade entre diferentes sistemas. De acordo com [Pautasso *et al.*, 2008],

os serviços REST caracterizam-se pela simplicidade, flexibilidade e escalabilidade, sendo amplamente empregados no desenvolvimento de aplicações distribuídas.

2.2 Segurança, Persistência de Dados e Implantação

A autenticação dos usuários foi implementada com base no padrão JSON Web Token (JWT), utilizando cookies HTTPS para armazenamento e validação dos tokens em requisições protegidas. Essa abordagem permite a implementação de uma autenticação stateless, eliminando a necessidade de manutenção de sessões no servidor e contribuindo para a escalabilidade da aplicação. Além disso, o emprego de mecanismos de autenticação baseados em tokens favorece a segurança e a interoperabilidade em aplicações web modernas. De acordo com [Camenisch and Lehmann, 2017], sistemas modernos de gerenciamento de identidades devem adotar mecanismos que permitam a autenticação e o compartilhamento seguro de informações entre diferentes entidades.

As informações relacionadas aos usuários, imagens enviadas, plantas, doenças, diagnósticos e textos são armazenadas em um banco de dados relacional PostgreSQL. O acesso aos dados foi realizado por intermédio do framework SQLAlchemy, empregando a abordagem Object Relational Mapping (ORM), enquanto o versionamento do esquema do banco foi conduzido por meio da ferramenta Alembic. A utilização do padrão ORM possibilita estabelecer uma camada de abstração entre os objetos da aplicação e as estruturas relacionais do banco de dados, favorecendo a reutilização de código, a manutenção do sistema e a redução da complexidade associada à manipulação direta de consultas SQL. De acordo com [Ambler, 1997], o mapeamento objeto-relacional constitui uma estratégia que facilita a integração entre o paradigma orientado a objetos e os sistemas de gerenciamento de bancos de dados relacionais.

Com o objetivo de garantir a padronização do ambiente de desenvolvimento e facilitar a implantação da aplicação em diferentes plataformas computacionais, foi empregada a tecnologia Docker. Essa abordagem permite encapsular todos os componentes necessários para execução do sistema, proporcionando maior portabilidade e reprodutibilidade dos experimentos. Segundo [Merkel, 2014], os contêineres Docker oferecem ambientes leves e consistentes para o desenvolvimento e implantação de aplicações distribuídas.

2.3 Interface Web e Comunicação Cliente-Servidor

A camada de front-end foi desenvolvida utilizando HTML5, CSS3 e JavaScript, integrados ao backend implementado em FastAPI. O sistema contempla funcionalidades de autenticação, cadastro de usuários, envio e pré-visualização de imagens, visualização dos diagnósticos, textos complementares e a consulta ao histórico das análises, além de interface responsiva para diferentes dispositivos [Frain, 2015].

Os recursos de interatividade foram implementados em JavaScript nativo (Vanilla JavaScript), incluindo manipulação dinâmica do DOM, validação de formulários, gerenciamento de sessões e atualização assíncrona das informações exibidas. A comunicação entre a interface e a API ocorre por meio de requisições HTTPS utilizando a API Fetch e troca de dados

em formato JSON, proporcionando maior fluidez na interação com o sistema [Flanagan, 2011; Zaidman *et al.*, 2013].

O gerenciamento das informações de sessão no lado do cliente foi realizado por meio da Web Storage API, permitindo a persistência temporária de dados e o controle de acesso às funcionalidades restritas da plataforma [Nikiforakis *et al.*, 2013].

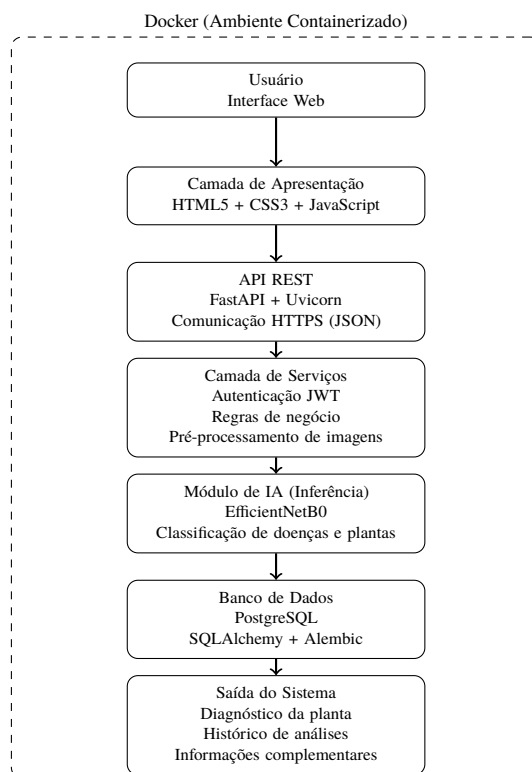


Figura 1. Arquitetura geral do sistema com execução containerizada em Docker.

2.4 Base de Dados e Ambiente Experimental

Para o treinamento dos modelos de classificação foi utilizada a base PlantVillage [Hughes and Salathé, 2015], que também está disponível no Kaggle [Raschka *et al.*, 2020], é amplamente empregada em pesquisas relacionadas à detecção automática de doenças em plantas. O conjunto de dados contém 54.309 imagens de folhas pertencentes a diferentes culturas agrícolas, incluindo amostras saudáveis e amostras afetadas por doenças causadas por fungos, bactérias e vírus. Segundo [Hughes and Salathé, 2015], a construção dessa base teve como objetivo fornecer um repositório aberto e padronizado para o desenvolvimento de sistemas automatizados de diagnóstico agrícola.

Os dados foram reorganizados em dois conjuntos independentes. O primeiro foi destinado ao treinamento de um modelo específico para identificação da espécie vegetal, contemplando 14 culturas presentes na base original. O segundo conjunto foi empregado no treinamento de um modelo especializado na identificação das doenças, considerando 38 categorias distintas. Essa estratégia de classificação em múltiplos estágios foi adotada com o objetivo de reduzir inconsistências entre planta e doença identificadas, sendo compatível com abordagens hierárquicas utilizadas em sistemas de classifica-

ção agrícola [Pacal *et al.*, 2024].

O treinamento foi realizado na plataforma Google Colab, utilizando GPUs NVIDIA T4 para acelerar as etapas de aprendizagem. Os conjuntos de dados foram armazenados no Google Drive e carregados dinamicamente durante a execução dos experimentos. Conforme [Carneiro *et al.*, 2018], o Google Colab disponibiliza recursos computacionais de alto desempenho, tornando-se uma alternativa amplamente utilizada em pesquisas envolvendo Aprendizado de Máquina e Deep Learning.

2.5 Treinamento e Otimização

Os modelos de classificação foram implementados utilizando a biblioteca TensorFlow/Keras, adotando-se a arquitetura EfficientNetB0, proposta por [Tan and Le, 2019]. Essa arquitetura emprega o conceito de escalonamento composto, permitindo balancear profundidade, largura e resolução da rede neural, alcançando elevado desempenho em tarefas de classificação de imagens com reduzido custo computacional.

Os dados foram organizados em uma estrutura de diretórios compatível com a função *image_dataset_from_directory* da biblioteca TensorFlow/Keras, a qual permite o carregamento automático das imagens a partir de pastas pré-definidas. Foi adotada a estratégia de particionamento *hold-out*, na qual o conjunto de dados tanto de plantas quanto de doenças é dividido previamente em dois subconjuntos independentes: treinamento (*train*) e validação (*valid*).

Nesse contexto, a separação não é realizada dinamicamente durante a execução do código, mas sim definida na própria organização da base de dados em diretórios distintos. Essa abordagem garante que a divisão entre os conjuntos permaneça fixa e consistente entre diferentes execuções dos experimentos, contribuindo para a reprodutibilidade dos resultados.

A divisão adotada mantém 80% das amostras para treinamento e 20% para validação. O conjunto de treinamento é utilizado exclusivamente para o ajuste dos pesos da rede neural, enquanto o conjunto de validação é empregado apenas para monitoramento do desempenho do modelo em dados não vistos, sem influência direta no processo de otimização.

Além disso, foi definido um valor fixo de *seed* (*seed*=42) durante o carregamento dos dados, de modo a garantir reprodutibilidade na ordenação e amostragem das imagens. Dessa forma, minimizam-se variações entre execuções e assegura-se maior confiabilidade na comparação dos resultados experimentais.

As imagens foram redimensionadas para 224×224 pixels e processadas em lotes de 16 amostras, utilizando-se a mesma etapa de pré-processamento empregada durante o treinamento. O desempenho foi mensurado por meio da métrica de acurácia frequentemente utilizadas em problemas de classificação [Yosinski *et al.*, 2014].

Com o objetivo de aumentar a variabilidade dos dados e reduzir o sobreajuste, foram aplicadas técnicas de aumento de dados (*Data Augmentation*), incluindo rotações aleatórias e inversões horizontais das imagens. Além disso, foi utilizado o pré-processamento nativo da EfficientNetB0 antes do fornecimento das amostras ao modelo [Shorten and Khoshgoufar, 2019; Tan and Le, 2019].

O treinamento foi realizado em duas etapas. Inicial-

mente, empregou-se a técnica de aprendizado por transferência (*Transfer Learning*), mantendo congeladas as camadas convolucionais previamente treinadas na base ImageNet e ajustando apenas as camadas responsáveis pela classificação. Posteriormente, foi realizado o processo de *Fine-Tuning*, liberando parte das camadas profundas da rede para atualização dos pesos com uma taxa de aprendizado reduzida. Essas estratégias possibilitam adaptar os conhecimentos previamente adquiridos ao domínio específico do problema, favorecendo a capacidade de generalização do modelo [Yosinski *et al.*, 2014; Kornblith *et al.*, 2019].

Para reduzir o risco de sobreajuste e tornar o treinamento mais estável, foram utilizados os mecanismos de parada antecipada (*Early Stopping*) e ajuste adaptativo da taxa de aprendizado (*ReduceLROnPlateau*).

Ao final do processo, foram obtidos dois modelos independentes, armazenados no formato *.keras*, juntamente com arquivos JSON contendo os nomes das classes. Um dos modelos foi destinado à identificação da espécie vegetal e o outro à classificação das doenças, sendo posteriormente integrados ao sistema por meio de uma estratégia de inferência em múltiplos estágios.

Tabela 2. Principais hiperparâmetros utilizados no treinamento dos modelos.

Parâmetro	Valor
Arquitetura	EfficientNetB0
Tamanho das imagens	224×224
Batch size	16
Otimizador	Adam
Taxa de aprendizado (fase 1)	10^{-3}
Taxa de aprendizado (fase 2)	10^{-5}
Função de perda	Sparse Categorical Crossentropy
Épocas (fase 1)	12
Épocas (fase 2)	15
Data Augmentation	Rotação e espelhamento horizontal
Early Stopping	Sim
ReduceLROnPlateau	Sim
Hardware	GPU NVIDIA T4
Ambiente	Google Colab

2.6 Processo de Inferência e Cálculo da Confiança

Durante a etapa de inferência, a imagem enviada pelo usuário é inicialmente submetida a um processo de pré-processamento compatível com a arquitetura *EfficientNetB0*, incluindo redimensionamento para 224×224 pixels e normalização por meio da função *preprocess_input* da API do TensorFlow/Keras, garantindo compatibilidade com o domínio de entrada utilizado no treinamento.

Em seguida, a imagem é encaminhada simultaneamente para dois modelos independentes: um modelo responsável pela classificação da espécie vegetal e outro especializado na identificação da doença. Ambos os modelos produzem vetores de probabilidade obtidos por meio da função *softmax*, representando a distribuição de confiança entre as classes possíveis.

Para aumentar a consistência das predições, o sistema aplica uma estratégia de inferência baseada em compatibilidade biológica entre planta e doença. As classes de doenças seguem uma estrutura hierárquica no formato

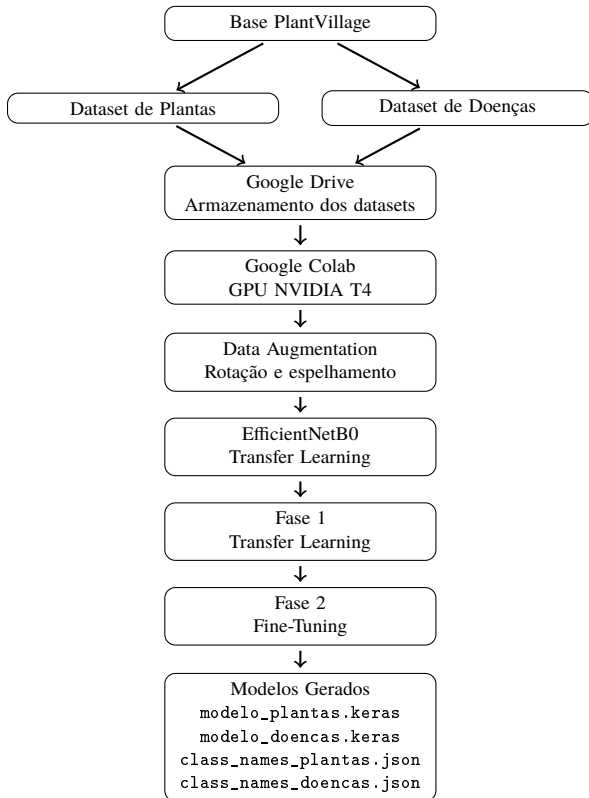


Figura 2. Pipeline de treinamento dos modelos de Inteligência Artificial.

Planta___Doença, permitindo o mapeamento direto entre os dois modelos. A partir desse mapeamento, é estabelecida uma relação de compatibilidade entre as saídas dos dois modelos, permitindo avaliar a consistência biológica entre planta e doença durante o processo de seleção da predição final.

O processo de seleção do par planta-doença é realizado por meio de uma estratégia *top-k*. São consideradas as três classes com maior probabilidade no modelo de plantas e as cinco classes mais prováveis no modelo de doenças. Para cada candidato, é calculado um *score* composto pela combinação ponderada das probabilidades:

$$Score = 0,6 \cdot P_{doena} + 0,4 \cdot P_{planta} + B \quad (1)$$

em que B representa um bônus adicional de consistência, aplicado quando a planta associada à doença candidata está entre as três classes mais prováveis do modelo de plantas.

O par final é definido como aquele que apresenta o maior *score* entre todas as combinações avaliadas. Caso nenhum par compatível seja encontrado, o sistema recorre a uma estratégia de *fallback* baseada na maior probabilidade global do modelo de doenças, associada à melhor correspondência disponível no modelo de plantas.

Casos específicos são tratados por regras adaptativas. Para culturas com baixa representatividade no conjunto de dados, como *Blueberry*(Mirtilo), *Raspberry*(Framboesa), *Orange*(laranja), *Cherry*(Cereja) e *Soybean*(Soja), a confiança final é obtida diretamente a partir da probabilidade produzida pelo modelo de doenças, reduzindo a influência do classificador de plantas. Já para a cultura *Tomato*, são aplicados ajustes adicionais baseados na distribuição das probabilidades, com análise de múltiplas classes candidatas, visando reduzir ambiguidades entre doenças visualmente semelhantes.

Além disso, heurísticas específicas são utilizadas para

corrigir classificações instáveis, como o caso de predições incorretas de *healthy* quando há forte evidência de outras doenças no vetor de saída.

Após a definição do par planta-doença, o nome da doença é normalizado e o resultado final é representado no formato *planta___doença*. A confiança final da predição é estimada de acordo com a disponibilidade de dados para cada cultura.

(i) Para culturas com poucos dados, utiliza-se diretamente a probabilidade do modelo de doenças;

(ii) Para as demais culturas, aplica-se uma combinação ponderada entre os dois modelos:

$$Confiana_{final} = 0,65 \cdot P_{doena} + 0,35 \cdot P_{planta} \quad (2)$$

Essa abordagem híbrida permite integrar informações complementares dos dois classificadores, aumentando a robustez do sistema, reduzindo inconsistências entre saídas independentes e melhorando a confiabilidade das predições finais.

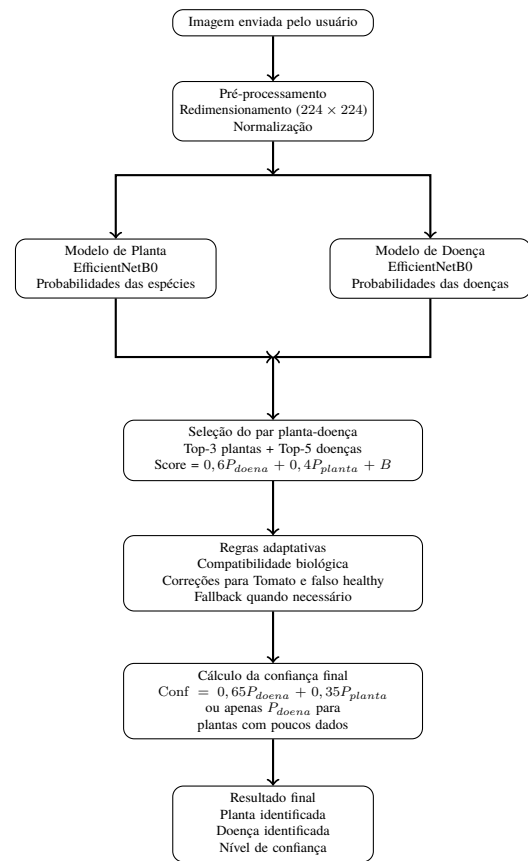


Figura 3. Fluxo do processo de inferência e cálculo da confiança do sistema proposto.

2.7 Integração com Modelos de Linguagem

Além do processo de classificação das doenças, foi implementado um módulo de geração automática de descrições textuais complementares associadas aos resultados obtidos pelo sistema. Esse módulo tem como objetivo fornecer informações educativas sobre a planta e a possível condição fitossanitária identificada, auxiliando o usuário na compreensão do diagnóstico produzido pelos modelos de Inteligência Artificial.

Para essa finalidade, foram utilizados modelos de linguagem acessados por meio de uma API de terceiros (Open-

Router), que atua como intermediador para a utilização de modelos de linguagem de grande porte. Essa abordagem permite a geração de textos explicativos em linguagem natural, sem a necessidade de gerenciamento direto da infraestrutura do modelo.

As respostas geradas são estruturadas de forma a apresentar descrições gerais sobre a doença detectada, incluindo características visuais comuns, possíveis causas biológicas e condições ambientais associadas ao seu desenvolvimento.

É importante destacar que o sistema não realiza a prescrição de tratamentos de intervenção química de agrotóxicos. Essa decisão foi adotada para evitar riscos decorrentes de possíveis erros de classificação, uma vez que diagnósticos incorretos poderiam levar a ações inadequadas e impactos ambientais indesejados.

Dessa forma, o módulo é restrito à geração de informações de caráter educativo e informativo, não devendo ser interpretado como substituto de orientação técnica especializada. Conforme descrito por [Yang *et al.*, 2024], modelos de linguagem de grande porte apresentam elevada capacidade de geração de texto contextualizado, sendo amplamente utilizados em aplicações de apoio à comunicação e disseminação de conhecimento.

3 Resultados e Discussão

3.1 Avaliação do Treinamento e Validação

A avaliação dos modelos foi realizada utilizando o conjunto de validação disponibilizado pela base PlantVillage, seguindo a estratégia de divisão fixa do tipo *Hold-out*, na qual 80% das imagens foram destinadas ao treinamento e os 20% restantes à validação. Essa abordagem permite estimar a capacidade de generalização do modelo em amostras não utilizadas durante o processo de aprendizado, reduzindo a possibilidade de superestimação do desempenho.

Os resultados obtidos demonstram elevada capacidade de classificação para ambas as tarefas propostas. Na primeira fase, *Transfer Learning* alcançou acurácia de 96,85%, enquanto na segunda fase o *Fine-Tuning* atingiu 98,81%. Esses resultados sugerem que a combinação da arquitetura EfficientNetB0 com as técnicas de *Transfer Learning*, *Data Augmentation*, regularização e refinamento dos pesos foi capaz de extrair características discriminativas dos padrões visuais presentes nas imagens da base PlantVillage.

As Figuras 4 e 5 apresentam a evolução da acurácia e da função de perda durante a primeira fase do treinamento. Observa-se crescimento gradual da acurácia, acompanhado pela redução consistente da função de perda, indicando convergência estável do processo de otimização. Além disso, as curvas de treinamento e validação permanecem próximas ao longo das épocas, sem oscilações acentuadas, sugerindo boa capacidade de generalização e o mecanismo de Early Stopping foi capaz de interromper o treinamento antes que surgissem indícios relevantes de sobreajuste (*overfitting*).

Outro aspecto observado é que a acurácia de validação permaneceu superior à acurácia de treinamento durante praticamente toda a primeira fase. Esse comportamento é esperado quando técnicas de regularização, como *Dropout* e *Data Augmentation*, são aplicadas exclusivamente durante o treinamento. Como essas técnicas tornam as amostras de

treinamento mais desafiadoras, o modelo tende a apresentar desempenho ligeiramente inferior nesse conjunto, enquanto o conjunto de validação permanece composto por imagens originais, resultando em métricas superiores durante a avaliação.

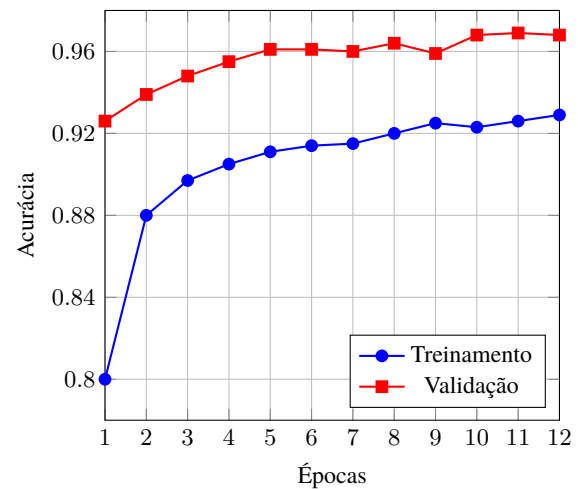


Figura 4. Evolução da acurácia durante o treinamento da Fase 1.

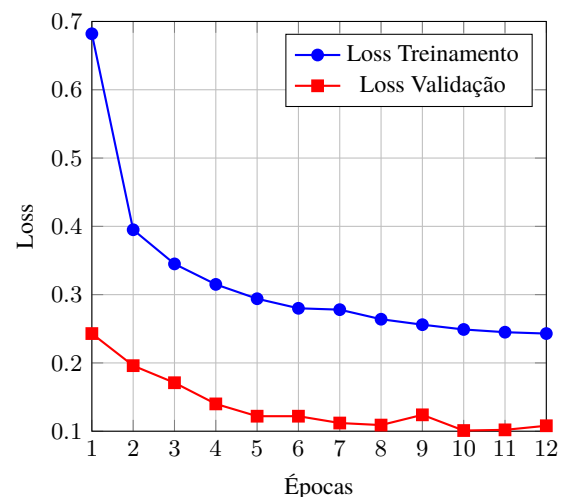


Figura 5. Evolução da função de perda durante o treinamento da Fase 1.

As Figuras 6 e 7 mostram a evolução das métricas durante a etapa de *Fine-Tuning*. Os resultados indicam que tanto a acurácia quanto a função de perda continuam evoluindo de maneira consistente após o desbloqueio das camadas profundas da EfficientNetB0, demonstrando que o refinamento dos pesos permitiu especializar a rede para o domínio de doenças foliares sem comprometer sua estabilidade. A acurácia de validação aumentou progressivamente de 94,88% na primeira época para 98,81% na 14ª época, enquanto a perda de validação foi reduzida de 0,1449 para 0,0369.

A proximidade entre as curvas de treinamento e validação durante o *Fine-Tuning* indica que a atualização dos pesos ocorreu de maneira controlada, sem degradação da capacidade de generalização. Embora a perda de treinamento permaneça superior à perda de validação durante toda essa etapa, esse comportamento continua sendo explicado pelo uso das técnicas de regularização e aumento de dados aplicadas apenas ao conjunto de treinamento. Além disso, nota-se

que a perda de validação praticamente estabiliza nas últimas épocas, indicando que o modelo atingiu um ponto próximo da convergência.

A melhor configuração foi obtida na 14ª época, quando a acurácia de validação atingiu 98,81% e a função de perda alcançou seu menor valor (0,0369). Na época seguinte, verificou-se uma pequena redução na acurácia de validação e um discreto aumento da perda, motivo pelo qual o mecanismo de *Early Stopping* restaurou automaticamente os pesos da melhor época. Esse comportamento demonstra que o critério de parada adotado foi eficiente para evitar o início do sobreajuste e preservar o modelo de melhor desempenho.

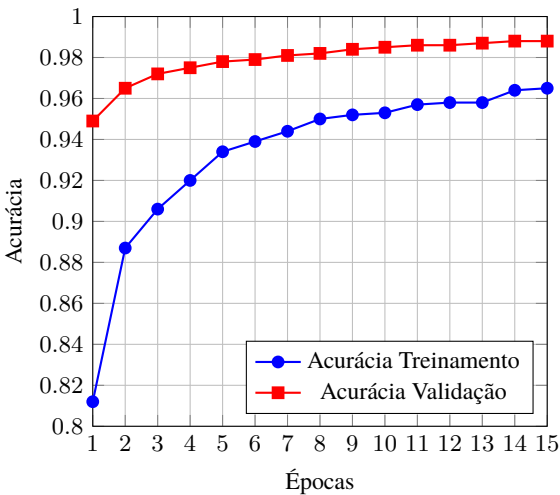


Figura 6. Evolução da acurácia durante o fine-tuning (Fase 2).

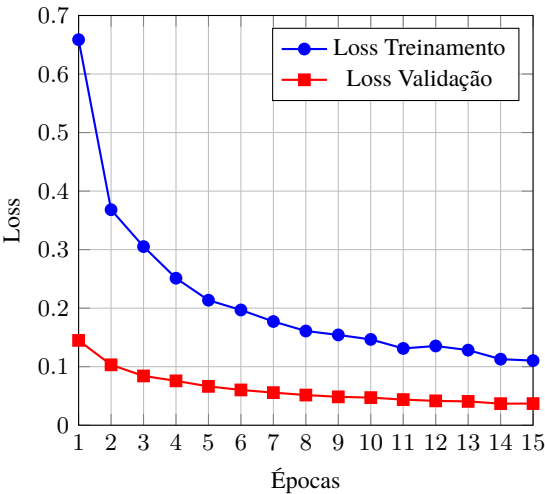


Figura 7. Evolução da função de perda durante o Fine-Tuning (Fase 2).

A Tabela 3 resume os resultados obtidos nas duas etapas de treinamento. O processo de *Fine-Tuning* elevou a acurácia global de 96,85% para 98,81%, correspondendo a um ganho aproximado de dois pontos percentuais. Embora esse incremento pareça modesto em termos absolutos, ele representa uma redução expressiva da taxa de erro do modelo, fator particularmente relevante em aplicações de diagnóstico automático, nas quais pequenas melhorias na precisão podem reduzir significativamente a ocorrência de classificações incorretas.

Esse ganho pode ser explicado pelo fato de que, na primeira etapa, apenas as camadas finais da rede foram treinadas, mantendo congelados os pesos previamente aprendidos na base ImageNet. Embora essas padrões visuais sejam eficientes para tarefas gerais de visão computacional, elas não capturam integralmente as características específicas das doenças foliares. Durante o *Fine-Tuning*, a atualização controlada das camadas profundas permitiu adaptar essas representações às particularidades da base PlantVillage, refinando a extração de características relacionadas à textura, coloração, formato e distribuição das lesões presentes nas folhas.

Tabela 3. Comparação de desempenho entre as fases de treinamento.

Modelo	Acurácia (%)
Transfer Learning (Fase 1)	96,85
Fine-tuning (Fase 2)	98,81

Os resultados obtidos estão em conformidade com estudos recentes que demonstram a elevada eficiência da arquitetura EfficientNet em tarefas de classificação de doenças em plantas Tan and Le [2019]. Da mesma forma, corroboram revisões da literatura que apontam o *Transfer Learning* e o *Fine-Tuning* como estratégias capazes de melhorar significativamente o desempenho de modelos de aprendizado profundo em aplicações agrícolas, principalmente quando comparadas ao treinamento realizado do zero Pacal *et al.* [2024]; Kornblith *et al.* [2019].

De maneira geral, a análise conjunta das curvas de acurácia e perda demonstra que o processo de treinamento apresentou convergência estável, elevada capacidade de generalização e o mecanismo de *Early Stopping* foi capaz de interromper o treinamento antes que surgissem indícios relevantes de sobreajuste. Assim, os resultados obtidos indicam que a metodologia proposta produziu modelos robustos e adequados para a identificação automática de espécies vegetais e doenças foliares na base PlantVillage, fornecendo evidências da eficácia da estratégia baseada em EfficientNetB0 associada ao *Transfer Learning* e ao *Fine-Tuning*.

3.2 Avaliação do Desempenho por Classe na Plataforma

Além da avaliação realizada sobre validação do treinamento, foi conduzido um experimento complementar para avaliar com objetivo analisar o comportamento da plataforma utilizando dez imagens representativas por classe, selecionadas de forma independente. Para cada classe foram contabilizados o número de acertos, erros e a confiança média das predições corretas, permitindo identificar quais doenças apresentaram maior facilidade ou dificuldade de reconhecimento.

A Tabela 4 apresenta os resultados obtidos para as 38 classes avaliadas. No total, foram analisadas 380 imagens, das quais 313 foram classificadas corretamente e 67 foram classificadas incorretamente. A confiança média das classificações foi de 89,47%, indicando que, quando o modelo identifica corretamente uma classe, as probabilidades atribuídas às predições tendem a ser elevadas.

Observa-se que diversas classes apresentaram desempenho perfeito durante essa avaliação. Culturas como tomate, maçã, cereja, morango, soja, mirtilo, uva saudável e pimentão

saudável obtiveram 100% de acertos, com níveis de confiança próximos de 100%. Esse comportamento sugere que o modelo foi capaz de aprender representações altamente discriminativas para essas doenças, mesmo em uma arquitetura relativamente compacta como a EfficientNetB0.

Entre as classes do tomateiro, que representam a maior parcela da base PlantVillage, apresentaram 100% de acerto na amostra avaliada, incluindo saudável, Mancha Bacteriana, Queima Precoce, Requeima, Mancha de Septória, Ácaro-rajado, Mancha-Alvo, Mofo das Folhas, Vírus do Mosaico e Vírus do Enrolamento Amarelo das Folhas. Esses resultados sugerem que o elevado número de amostras disponíveis para essa cultura favoreceu a aprendizagem das características específicas de suas doenças.

Por outro lado, algumas classes apresentaram desempenho de acerto inferior às demais, destacando-se principalmente Requeima da Batata (48,12%), Batata Saudável (54,50%), Queima Precoce da Batata (60,98%), Huanglongbing da Laranjeira (60,99%), Ferrugem Comum do Milho (61%), Mancha Bacteriana da Pimenteira (66,90%) e Mancha Foliar da Videira (67,50%). Nessas categorias, observou-se redução tanto da taxa de acerto quanto da confiança média das classificações, indicando maior dificuldade do modelo em distinguir padrões visuais semelhantes.

Essa diferença de desempenho pode ser explicada por diversos fatores. O primeiro está relacionado à elevada similaridade visual entre determinadas doenças foliares, especialmente nos estágios iniciais de desenvolvimento, quando alterações de coloração, necroses e manchas apresentam características bastante semelhantes entre espécies distintas. Em consequência, pequenas diferenças de textura ou distribuição das lesões tornam-se decisivas para a classificação automática, aumentando a probabilidade de confusão entre classes, conforme observado em estudos recentes sobre diagnóstico de doenças em plantas [Pacal *et al.*, 2024; Liu and Wang, 2021].

Outro fator importante refere-se ao desbalanceamento da base PlantVillage. Embora todas as classes possuam quantidade suficiente de imagens para treinamento, existe predominância de determinadas culturas, principalmente o tomateiro, que concentra um número significativamente maior de amostras do que outras culturas. Essa distribuição desigual favorece o aprendizado das classes majoritárias, fazendo com que a rede neural desenvolva representações mais robustas para essas categorias durante o processo de otimização.

Durante a inspeção qualitativa das predições incorretas, observou-se ainda que parte dos erros ocorreu justamente por associações com doenças do tomateiro, mesmo quando não tem muitas similaridades. Esse comportamento sugere que, diante de imagens ambíguas, o modelo tende a favorecer classes pertencentes à cultura com maior representatividade na base de treinamento, fenômeno frequentemente observado em modelos treinados sobre conjuntos de dados desbalanceados.

Apesar dessas limitações, os resultados demonstram que a maior parte das classes apresentou elevado desempenho e alta confiança nas classificações, indicando a capacidade do modelo em extrair características discriminativas relevantes para diferentes espécies vegetais. Além disso, a estratégia proposta neste trabalho, baseada na utilização de dois modelos independentes, um para identificação da espécie vegetal e outro para classificação da doença, associada ao mecanismo

Tabela 4. Desempenho por classe - Confiança média das predições

Classe	Total	Acertos	Erros	Média (%)
Sarna da maçã (Apple Scab)	10	10	0	99.95
Podridão negra da maçã (Apple Black Rot)	10	8	2	87.00
Ferrugem do cedro-maciceira (Cedar Apple Rust)	10	10	0	99.99
Maçã saudável (Apple Healthy)	10	10	0	99.83
Mirtilo saudável (Blueberry Healthy)	10	10	0	99.62
Cereja saudável (Cherry Healthy)	10	10	0	100.00
Oídio da cerejeira (Cherry Powdery Mildew)	10	10	0	100.00
Mancha de Cercospora do milho (Corn Cercospora Leaf Spot / Gray Leaf Spot)	10	9	1	91.03
Ferrugem comum do milho (Corn Common Rust)	10	4	6	61.00
Milho saudável (Corn Healthy)	10	6	4	74.00
Queima das folhas do milho (Corn Northern Leaf Blight)	10	7	3	82.40
Podridão negra da videira (Grape Black Rot)	10	6	4	72.84
Esca da videira (Grape Black Measles)	10	9	1	93.49
Uva saudável (Grape Healthy)	10	10	0	99.98
Mancha foliar da videira (Grape Leaf Blight)	10	5	5	67.50
Huanglongbing da laranjeira (Orange Huanglongbing)	10	4	6	60.99
Mancha bacteriana do pessegueiro (Peach Bacterial Spot)	10	7	3	80.49
Pêssego saudável (Peach Healthy)	10	9	1	93.50
Mancha bacteriana da pimenteira (Pepper Bacterial Spot)	10	5	5	66.99
Pimentão saudável (Pepper Healthy)	10	10	0	99.87
Queima precoce da batata (Potato Early Blight)	10	4	6	60.98
Batata saudável (Potato Healthy)	10	3	7	54.5
Requeima da batata (Potato Late Blight)	10	2	8	48.12
Framboesa saudável (Raspberry Healthy)	10	9	1	93.50
Soja saudável (Soybean Healthy)	10	10	0	100.00
Oídio da abóbora (Squash Powdery Mildew)	10	8	2	87.00
Morango saudável (Strawberry Healthy)	10	10	0	100.00
Queimadura foliar do morango (Strawberry Leaf Scorch)	10	8	2	80.50
Mancha bacteriana do tomateiro (Tomato Bacterial Spot)	10	10	0	99.90
Queima precoce do tomateiro (Tomato Early Blight)	10	10	0	94.60
Tomate saudável (Tomato Healthy)	10	10	0	99.96
Requeima do tomateiro (Tomato Late Blight)	10	10	0	92.20
Mofo das folhas do tomateiro (Tomato leaf Mold)	10	10	0	99.93
Mancha de Septória do tomateiro (Tomato Septoria Leaf Spot)	10	10	0	99.29
Ácaro-rajado do tomateiro (Tomato Spider Mites Two-Spotted Spider Mite)	10	10	0	96.78
Mancha-alvo do tomateiro (Tomato Target Spot)	10	10	0	98.88
Vírus do mosaico do tomateiro (Tomato Mosaic Virus)	10	10	0	99.94
Vírus do enrolamento amarelo das folhas do tomateiro (Tomato Yellow Leaf Curl Virus)	10	10	0	99.97
Total imagens	380	313	67	89.47

de compatibilidade biológica implementado na plataforma, contribuiu para reduzir inconsistências entre as predições e aumentar a confiabilidade do diagnóstico final apresentado ao usuário.

Em conjunto, esses resultados indicam que a metodologia proposta apresenta elevado potencial para aplicações práticas de diagnóstico fitossanitário, embora evidenciem oportunidades de aprimoramento futuro, como a utilização de técnicas de balanceamento de classes, treinamento com bases mais diversificadas e inclusão de imagens obtidas em condições reais de campo, reduzindo o impacto do desbalanceamento e aumentando a capacidade de generalização do sistema.

3.3 Avaliação da Plataforma

Além da avaliação dos modelos de Inteligência Artificial, foi realizada uma avaliação funcional da plataforma web desenvolvida, com o objetivo de verificar o correto funcionamento

e a integração entre os componentes da arquitetura proposta. Os testes abrangeram todo o fluxo de utilização do sistema, incluindo autenticação de usuários, processamento das imagens, comunicação entre o frontend, backend e modelos de aprendizado profundo, armazenamento das informações no banco de dados e consulta ao histórico de diagnósticos.

A autenticação representa a etapa inicial de utilização da plataforma. Conforme ilustrado na Figura 8, o sistema disponibiliza funcionalidades de cadastro e login utilizando autenticação baseada em *JSON Web Token* (JWT), permitindo o acesso seguro às funcionalidades restritas da aplicação. Após a validação das credenciais, o token gerado é utilizado para autenticar as requisições enviadas ao servidor, garantindo que apenas usuários autorizados possam acessar e registrar diagnósticos.

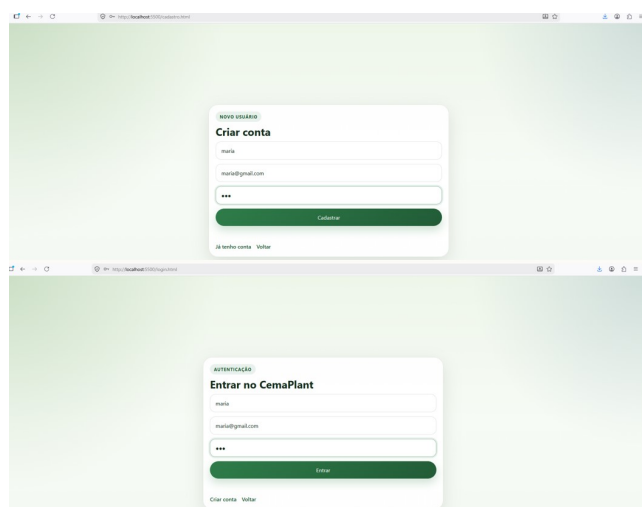


Figura 8. Tela de login e cadastro da plataforma.

Após a autenticação, o usuário é direcionado ao painel principal da plataforma, apresentado na Figura 9. Esse ambiente centraliza as principais funcionalidades do sistema, oferecendo acesso às ferramentas de diagnóstico, ao histórico de análises e aos demais módulos da aplicação. A interface foi desenvolvida com foco na simplicidade de navegação, reduzindo a quantidade de etapas necessárias para a execução das operações mais frequentes.

A Figura 10 apresenta a tela de seleção das culturas agrícolas disponíveis para análise. Essa organização permite ao usuário navegar de forma intuitiva entre as espécies cadastradas, facilitando a utilização da plataforma e tornando o processo de diagnóstico mais objetivo.

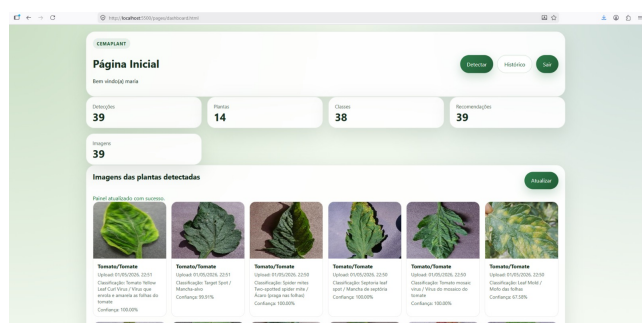


Figura 9. Tela inicial (dashboard) da plataforma.

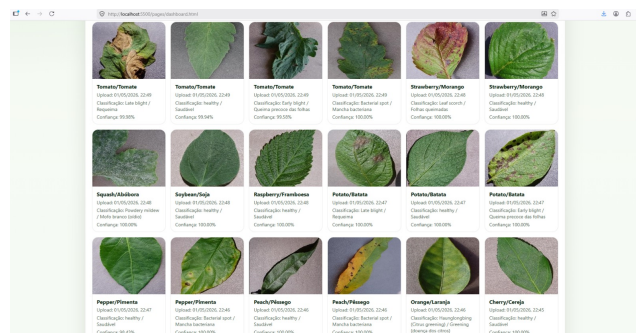


Figura 10. Dashboard com seleção de culturas agrícolas.

O envio das imagens é realizado por meio da interface ilustrada na Figura 11. Nessa etapa, o usuário seleciona uma fotografia da folha que será submetida aos modelos de Inteligência Artificial para processamento. Antes da análise, o sistema verifica se o arquivo enviado corresponde a uma imagem válida e, na interface, disponibiliza uma pré-visualização da imagem selecionada. Essas validações evitam o processamento de arquivos inválidos, reduzem a ocorrência de erros durante a execução do sistema e proporcionam uma melhor experiência de utilização.

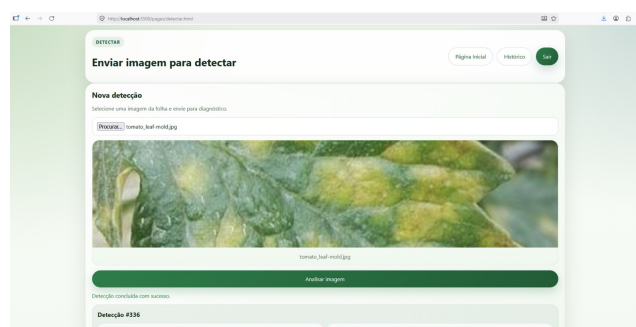


Figura 11. Tela de envio de imagens para análise.

Após o processamento, o sistema retorna os resultados da predição, apresentados nas Figuras 12 e 13. O sistema apresenta simultaneamente a espécie identificada, a doença detectada e o nível de confiança da classificação. A integração dessas informações com descrições textuais geradas automaticamente amplia o caráter interpretativo do diagnóstico e diferencia a plataforma de soluções que apresentam apenas o rótulo da classe prevista.

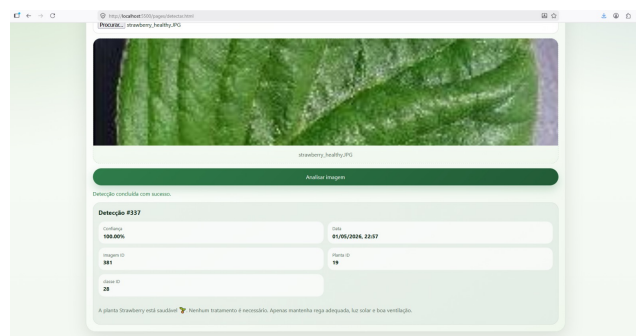


Figura 12. Resultado do diagnóstico indicando planta saudável.

Mesmo quando são utilizadas imagens que não pertencem à base PlantVillage, como ilustrado na Figura 13, o sistema apresentou uma classificação compatível com o estado visual da planta avaliada, embora com menor nível de confi-

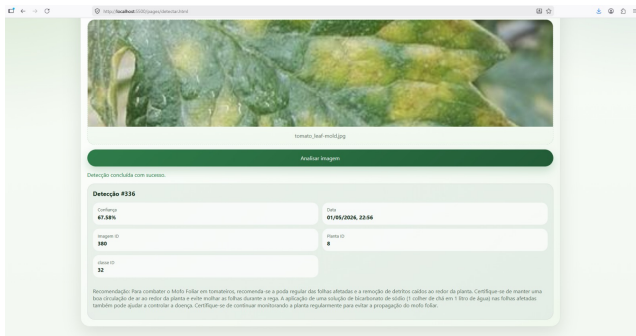


Figura 13. Resultado do diagnóstico indicando presença de doença com foto que não pertence a base de treinamento.

ança. Esse resultado sugere que os modelos não memorizaram apenas as imagens do conjunto de treinamento, apresentando capacidade de generalização para amostras inéditas.

Por fim, a Figura 14 apresenta a funcionalidade de histórico de diagnósticos. Nessa seção, o usuário pode consultar todas as análises previamente realizadas, visualizar novamente as imagens enviadas e acessar os respectivos resultados. Além de facilitar o acompanhamento contínuo das condições fitossanitárias das plantas monitoradas, essa funcionalidade garante a persistência dos dados e a rastreabilidade das inferências produzidas pelos modelos de Inteligência Artificial.

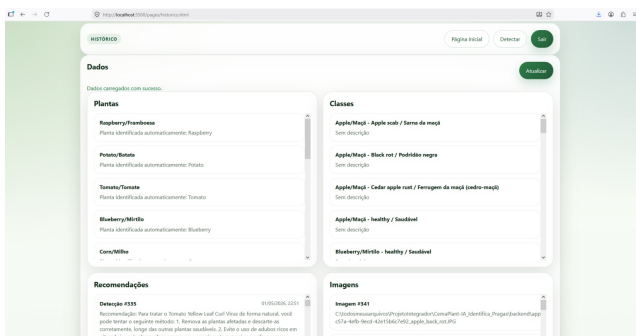


Figura 14. Tela de histórico das análises realizadas pelos usuários.

Durante os testes de validação da plataforma, todas as funcionalidades implementadas apresentaram comportamento consistente. O processo de autenticação, o envio das imagens, a comunicação entre o frontend e a API REST, a execução dos modelos de classificação, a geração dos resultados e o armazenamento das análises ocorreram sem falhas funcionais. Dessa forma, os experimentos confirmam que a arquitetura proposta integrou adequadamente os componentes de interface, serviços de backend, banco de dados e modelos de aprendizado profundo, disponibilizando uma solução capaz de realizar diagnósticos automatizados de doenças em plantas de forma rápida, organizada e acessível ao usuário.

3.4 Comparação com trabalhos relacionados

A comparação dos resultados obtidos neste trabalho com aqueles reportados na literatura deve ser realizada com cautela, uma vez que diferentes estudos empregam conjuntos de dados, protocolos experimentais, estratégias de particionamento, técnicas de pré-processamento e arquiteturas de redes neurais distintos. Esses fatores influenciam diretamente as métricas de desempenho, tornando inadequada a comparação baseada exclusivamente nos valores de acurácia.

Na base PlantVillage, utilizada neste trabalho, Mohanty *et al.* [2016] reportaram acurácia superior a 99% empregando redes neurais convolucionais profundas. Revisões recentes [Pacal *et al.*, 2024; Liu and Wang, 2021; Shoaib *et al.*, 2023] mostram que arquiteturas modernas, como EfficientNet, ResNet, DenseNet e Vision Transformers, frequentemente alcançam acurácias superiores a 98% quando avaliadas em bases de dados obtidas em condições controladas. Esses resultados demonstram que a classificação automática de doenças vegetais em ambientes controlados constitui um problema amplamente explorado, no qual diferentes arquiteturas apresentam desempenho bastante elevado.

Nesse contexto, o modelo proposto neste trabalho obteve acurácia de 98,81% na classificação da segunda fase utilizando a EfficientNetB0, resultado compatível com que foi reportado na literatura. Além do bom desempenho, a arquitetura adotada apresenta menor custo computacional quando comparada a modelos mais complexos, característica que favorece sua utilização em aplicações práticas e ambientes com recursos computacionais limitados [Tan and Le, 2019].

Entretanto, a principal contribuição deste trabalho não está apenas na obtenção de elevada acurácia, mas na arquitetura da solução desenvolvida. Diferentemente da maioria dos estudos analisados, que utilizam um único classificador para identificar simultaneamente a espécie vegetal e a doença, a abordagem proposta emprega dois modelos independentes: um destinado ao reconhecimento da espécie e outro à identificação da doença. As predições produzidas por esses modelos são posteriormente integradas por um mecanismo de verificação de compatibilidade biológica, responsável por validar se a doença identificada pode, de fato, ocorrer na espécie reconhecida. Essa estratégia reduz inconsistências entre as predições e aumenta a confiabilidade do diagnóstico final, representando uma contribuição prática pouco explorada nos trabalhos relacionados.

Outro diferencial da proposta refere-se ao desenvolvimento de uma plataforma web completa para disponibilização dos modelos de Inteligência Artificial. Enquanto grande parte dos trabalhos limita-se à avaliação experimental dos classificadores, a solução desenvolvida integra autenticação de usuários, armazenamento persistente dos diagnósticos, histórico de consultas, processamento automatizado das imagens, comunicação entre frontend e backend por meio de API REST e geração automática de descrições textuais utilizando modelos de linguagem. Dessa forma, o trabalho extrapola a etapa de desenvolvimento do modelo preditivo, oferecendo uma solução integrada voltada à utilização em cenários reais de apoio ao diagnóstico fitossanitário.

Apesar dos resultados promissores, as limitações observadas são consistentes com aquelas relatadas na literatura. Assim como em diversos estudos anteriores [Mohanty *et al.*, 2016; Pacal *et al.*, 2024; Liu and Wang, 2021], o treinamento foi realizado utilizando a base PlantVillage, composta predominantemente por imagens obtidas em condições controladas de iluminação, fundo homogêneo e baixa variabilidade ambiental. Essas características favorecem o desempenho dos modelos, mas não representam integralmente as condições encontradas em ambientes agrícolas reais.

Consequentemente, embora a plataforma tenha apresentado desempenho satisfatório inclusive em imagens externas à

base de treinamento, a avaliação em condições reais de campo ainda representa um desafio importante e uma oportunidade para trabalhos futuros. A utilização e bases de dados compostas por imagens capturadas em diferentes condições de iluminação, oclusão, fundo e estágios de desenvolvimento das plantas permitirá avaliar de forma mais abrangente a robustez e a capacidade de generalização da solução proposta.

3.5 Limitações do Estudo

Embora os resultados obtidos demonstrem desempenho satisfatório, algumas limitações devem ser consideradas. A principal refere-se à utilização da base PlantVillage, cujas imagens foram adquiridas em condições controladas de iluminação, fundo homogêneo e folhas isoladas. Embora adequadas para o treinamento dos modelos, essas condições não representam integralmente os cenários encontrados em ambientes agrícolas reais, nos quais fatores como iluminação variável, oclusões e diferentes estágios de desenvolvimento das plantas tornam a classificação mais desafiadora [Pacal *et al.*, 2024; Liu and Wang, 2021; Hughes and Salathé, 2015].

Outra limitação está relacionada ao desbalanceamento entre as classes da base de dados, o que pode favorecer culturas com maior número de amostras e influenciar o desempenho em espécies menos representadas.

Além disso, a avaliação foi realizada por meio de uma estratégia *hold-out* utilizando a própria PlantVillage. Assim, embora os resultados indiquem boa capacidade de generalização para imagens não utilizadas no treinamento, ainda são necessários experimentos com bases de dados obtidas em condições reais, bem como avaliações da plataforma com usuários finais, para validar sua aplicabilidade em cenários práticos.

4 Conclusão

Este trabalho apresentou o desenvolvimento de uma plataforma web baseada em Inteligência Artificial para a identificação automática de doenças em plantas a partir da análise de imagens de folhas. A solução proposta integra técnicas de Visão Computacional e Aprendizado Profundo em uma arquitetura cliente-servidor, combinando dois modelos especializados baseados na arquitetura EfficientNetB0, sendo um destinado à identificação da espécie vegetal e outro à classificação da doença associada.

Os resultados experimentais demonstraram a eficácia da abordagem proposta para o problema investigado. Os modelos alcançaram índices relevantes de acurácia nos conjuntos de validação, indicando que o uso de estratégias como *Transfer Learning*, *Fine-Tuning*, aumento de dados e mecanismos de regularização contribuiu significativamente para o aprendizado de representações discriminativas das imagens. Adicionalmente, a estratégia de inferência baseada na compatibilidade entre espécie vegetal e doença mostrou-se eficiente na redução de inconsistências entre as predições dos classificadores, aumentando a confiabilidade do diagnóstico final.

Outro aspecto relevante deste trabalho é a integração dos modelos de Inteligência Artificial em uma plataforma web completa, que contempla autenticação de usuários, armazenamento do histórico de diagnósticos, processamento automatizado de imagens e geração de descrições textuais

complementares. Dessa forma, a pesquisa ultrapassa a etapa de construção de modelos preditivos, oferecendo uma solução funcional e aplicada que aproxima os avanços da Inteligência Artificial de cenários reais de uso na agricultura digital.

Entretanto, algumas limitações devem ser consideradas na interpretação dos resultados. Os modelos foram treinados e avaliados utilizando a base PlantVillage, composta predominantemente por imagens obtidas em condições controladas. Assim, o desempenho observado pode não se manter integralmente em ambientes reais de cultivo, nos quais há maior variabilidade de iluminação, fundo, oclusões e qualidade das imagens.

Como trabalhos futuros, pretende-se ampliar a base de dados com imagens coletadas em campo, realizar avaliações com bases independentes, investigar arquiteturas mais recentes, como Vision Transformers e modelos híbridos, com o objetivo de aumentar a interpretabilidade das decisões dos modelos. Também pretende-se fazer um polimento no frontend da plataforma e expandir para abranger um número maior de culturas agrícolas e integrar mecanismos de monitoramento contínuo voltados ao apoio à agricultura de precisão.

Conclui-se, portanto, que a solução proposta constitui uma abordagem viável para o apoio ao diagnóstico fitossanitário, ao integrar modelos de aprendizado profundo, estratégias de validação das predições e uma plataforma web completa. Os resultados obtidos indicam que a arquitetura desenvolvida possui potencial para contribuir com aplicações de agricultura digital, oferecendo uma ferramenta capaz de auxiliar produtores e técnicos na identificação precoce de doenças em plantas.

Declarações complementares

Conflitos de interesse

Os autores declaram que não têm nenhum conflito de interesses.

Disponibilidade de dados e materiais

Os conjuntos de dados utilizados neste estudo são provenientes da base pública PlantVillage. Os códigos-fonte desenvolvidos durante esta pesquisa estão disponíveis em repositório público no GitHub, acessível em <https://github.com/MariaQuaresma/CemaPlant>

Outras informações relevantes

O desenvolvimento deste trabalho contou com o uso do ChatGPT como apoio à revisão textual e aprimoramento da redação científica. Todas as decisões técnicas, análises, resultados e a versão apresentada são de responsabilidade dos autores. Este trabalho não envolveu pesquisa com seres humanos ou animais.

Referências

- Ambler, S. W. (1997). Mapping objects to relational databases. AmbySoft Inc. White Paper. Version of August 17, 1997. Material adapted from *Building Object Applications That Work* (SIGS Books, 1997).
- Camenisch, J. and Lehmann, A. (2017). Privacy-preserving user-auditable pseudonym systems. *IEEE Transactions on Dependable and Secure Computing*, 16(1):146–159. DOI: 10.1109/TDSC.2017.2693980.
- Carneiro, T., Da Nóbrega, R. V. M., Nepomuceno, T., Bian, G.-B., De Albuquerque, V. H. C., and Rebouças Filho,

- P. P. (2018). Performance analysis of google colaboratory as a tool for accelerating deep learning applications. *IEEE Access*, 6:61677–61685. DOI: 10.1109/ACCESS.2018.2874767.
- Farias, K. and Lazzari, L. (2023). Event-driven architecture and rest architectural style: An exploratory study on modularity. *Journal of Applied Research and Technology*, 21(3):338–351. DOI: 10.22201/icat.24486736e.2023.21.3.1764.
- Flanagan, D. (2011). *JavaScript: The Definitive Guide: Activate Your Web Pages*. O'Reilly Media, Inc., Sebastopol, CA, 6 edition.
- Frain, B. (2015). *Responsive Web Design with HTML5 and CSS3*. Packt Publishing.
- Hughes, D. P. and Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. *arXiv preprint arXiv:1511.08060*.
- Kornblith, S., Shlens, J., and Le, Q. V. (2019). Do better imagenet models transfer better? In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2661–2671. DOI: 10.1109/CVPR.2019.00277.
- Liu, J. and Wang, X. (2021). Plant diseases and pests detection based on deep learning: A review. *Plant Methods*, 17:22. DOI: 10.1186/s13007-021-00722-9.
- Merkel, D. (2014). Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239):2. DOI: 10.5555/2600239.2600241.
- Mohanty, S. P., Hughes, D. P., and Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7:1419. DOI: 10.3389/fpls.2016.01419.
- Nikiforakis, N., Invernizzi, L., Kapravelos, A., Van Acker, S., Joosen, W., Kruegel, C., Piessens, F., and Vigna, G. (2013). Cookieless monster: Exploring the ecosystem of web-based device fingerprinting. In *2013 IEEE Symposium on Security and Privacy*, pages 541–555. IEEE. DOI: 10.1109/SP.2013.43.
- Pacal, I., Kunduracioglu, I., Alma, M. H., Deveci, M., Kadry, S., Nedoma, J., Slany, V., and Martinek, R. (2024). A systematic review of deep learning techniques for plant diseases. *Artificial Intelligence Review*, 57(11):304. DOI: 10.1007/s10462-024-10944-7.
- Pautasso, C., Zimmermann, O., and Leymann, F. (2008). Restful web services vs. big web services: Making the right architectural decision. In *Proceedings of the 17th International Conference on World Wide Web*, pages 805–814, New York, NY, USA. ACM. DOI: 10.1145/1367497.1367606.
- Quaresma, M. (2026). Cemaplant: Ia para identificação de doenças em plantas. <https://github.com/MariaQuaresma/CemaPlant>. Acesso em: 2 jun. 2026.
- Raschka, S., Patterson, J., and Nolet, C. (2020). Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence. *Information*, 11(4):193. DOI: 10.3390/info11040193.
- Shoaib, M., Shah, B., El-Sappagh, S., Ali, A., Ullah, A., Alenezi, F., Gechev, T., Hussain, T., and Ali, F. (2023). An advanced deep learning models-based plant disease detection: A review of recent research. *Frontiers in Plant Science*, 14:1158933. DOI: 10.3389/fpls.2023.1158933.
- Shorten, C. and Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):1–48. DOI: 10.1186/s40537-019-0197-0.
- Szeliski, R. (2022). *Computer Vision: Algorithms and Applications*. Springer Nature, 2 edition.
- Tan, M. and Le, Q. V. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pages 6105–6114.
- Yang, J., Jin, H., Tang, R., Han, X., Feng, Q., Jiang, H., Zhong, S., Yin, B., and Hu, X. (2024). Harnessing the power of llms in practice: A survey on chatgpt and beyond. *ACM Transactions on Knowledge Discovery from Data*, 18(6):1–32. DOI: 10.1145/3649506.
- Yosinski, J., Clune, J., Bengio, Y., and Lipson, H. (2014). How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems*, volume 27, pages 3320–3328.
- Zaidman, A., Matthijssen, N., Storey, M.-A., and Van Deursen, A. (2013). Understanding ajax applications by connecting client and server-side execution traces. *Empirical Software Engineering*, 18(2):181–218. DOI: 10.1007/s10664-012-9200-5.